

Lab Report: Object-Oriented Banking System

Java Implementation of Encapsulation, Inheritance & Polymorphism

1. Problem Statement / Question

Design and implement a Java program to simulate a basic banking system demonstrating Object-Oriented Programming principles. Create a base class **Account** containing private fields for account number and holder name, and a public balance. Implement appropriate constructors, getter methods, and functional methods such as `deposit()`, `withdraw()`, `calinterest()`, `transfer()`, and `show()`.

Derive two classes from Account:

- **SavingsAccount:** Implements a specific withdrawal logic checking for insufficient balance and calculates interest based on an interest rate.
- **CurrentAccount:** Implements a specific withdrawal logic considering an overdraft limit and applies a flat maintenance charge when calculating interest.

Create a main class to instantiate both account types, perform transactions (deposits, withdrawals, transfers, interest calculations), and display the final state.

2. Step-by-Step Process

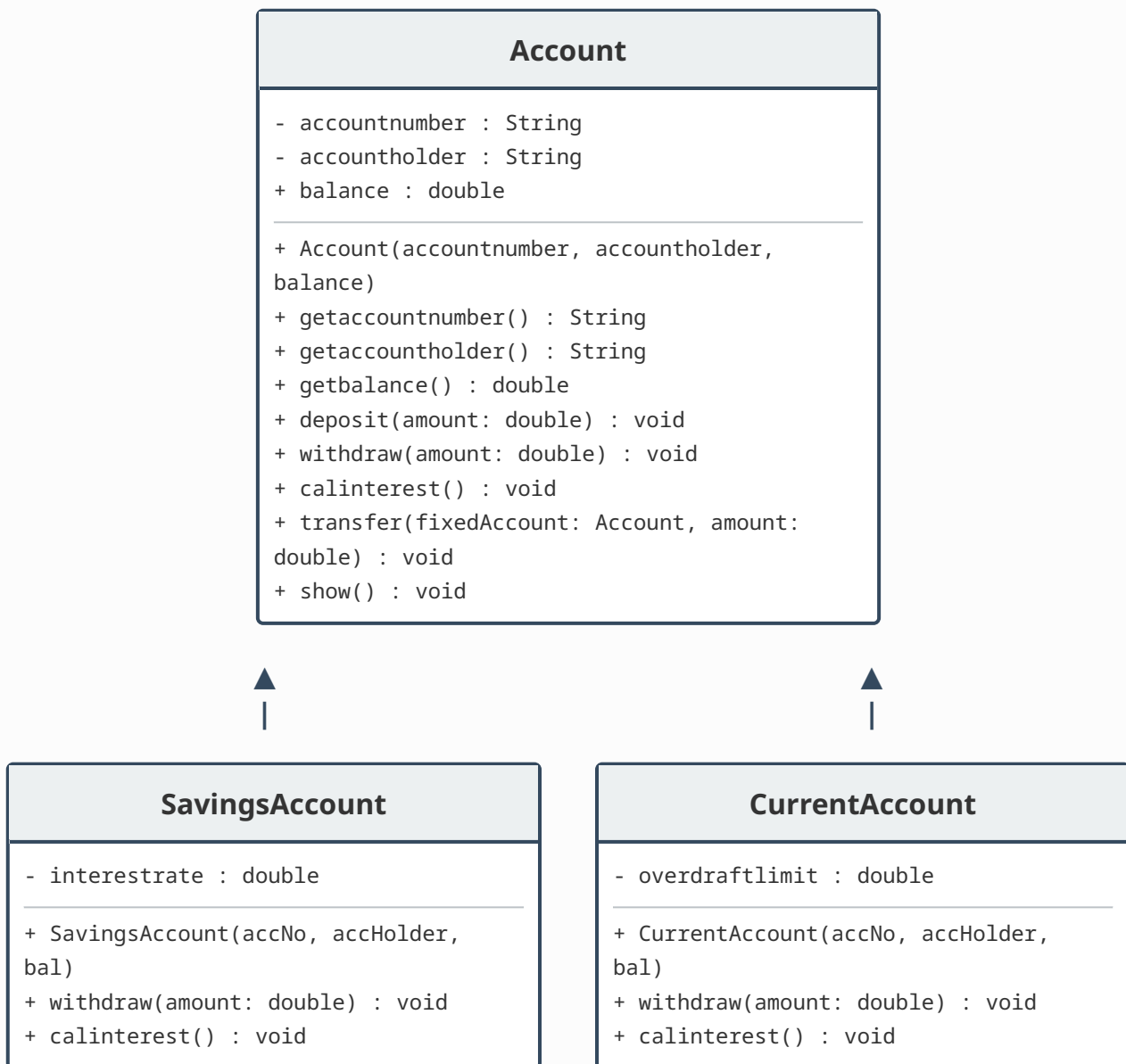
1. **Class Definition (Base Class):** Define the Account class with encapsulated variables (accountnumber, accountholder) and a balance variable.
2. **Constructor Initialization:** Use a parameterized constructor in Account to initialize the account details using the `this` keyword.
3. **Encapsulation:** Provide public getter methods to allow safe, read-only access to the private account details.
4. **Method Implementation:** Implement common methods like `deposit()` (adding to balance), `transfer()` (calling `withdraw` on self and `deposit` on the target account), and `show()`. Define placeholder or basic logic for `withdraw()` and `calinterest()`.
5. **Inheritance (SavingsAccount):** Create SavingsAccount extending Account. Use `super()` in the constructor to invoke the parent constructor. Override `withdraw()` to restrict negative balances and `calinterest()` to add percentage-based interest.
6. **Inheritance (CurrentAccount):** Create CurrentAccount extending Account. Use `super()` to initialize parent fields. Override `withdraw()` to allow dipping into an overdraft limit, and `calinterest()` to deduct/add a flat maintenance fee.
7. **Execution (Main Method):** In LabProgram1, instantiate the objects, invoke various transactions in sequence, and observe polymorphic behavior where the overridden methods of the specific object types are correctly invoked.

3. Algorithm

Algorithm for Bank Transactions:

1. **Start**
2. Create Current Account `ca` with initial balance 4500.
3. Create Savings Account `sa` with initial balance 5000.
4. Display initial details for `ca` and `sa`.
5. Invoke `ca.deposit(2000)`: Add 2000 to `ca`'s balance.
6. Invoke `ca.withdraw(2500)`: Check if `amount < balance + overdraft`. Since $2500 < 6500$, deduct 2500.
7. Invoke `ca.calinterest()`: Apply flat charge of 50.0 to `ca`'s balance.
8. Invoke `sa.withdraw(3000)`: Check if `amount < balance`. Since $3000 < 5000$, deduct 3000.
9. Invoke `sa.calinterest()`: Calculate interest and add to `sa`'s balance.
10. Invoke `sa.transfer(ca, 1000)`:
 - Withdraw 1000 from `sa`.
 - Deposit 1000 to `ca`.
11. Display final details for `ca` and `sa`.
12. **End**

4. UML Class Diagram



5. Source Code

```
class Account{
    private String accountnumber; // details are hide
    private String accountholder;
    public double balance;

    public Account(String accountnumber , String accountholder ,double balance){
        this.accountnumber = accountnumber; //using constructors for parameters passing
        this.accountholder = accountholder;
        this.balance = balance;
    }

    public String getaccountnumber() // doing encapsulation by setters and getters
    {
        return accountnumber;
    }

    public String getaccountholder(){
        return accountholder;
    }

    public double getbalance(){
        return balance;
    }

    void deposit (double amount){
        if(amount <= 0){
            System.out.println("deposit amount is greater than zero");
            return;
        }
        balance += amount;
        System.out.println(" the deposited amount is " + amount);
    }

    void withdraw (double amount){
    };

    void calinterest(){};

    void transfer(Account fixedAccount , double amount){
        System.out.println("transfer of "+amount+" from "+accountnumber+" to "+fixedAccount.getaccountnumber());
        this.withdraw(amount);
        fixedAccount.deposit(amount);
    }

    void show(){
        System.out.println("ACCOUNT NUMBER IS " + accountnumber);
        System.out.println("ACCOUNT HOLDER IS " + accountholder);
        System.out.println("BALANCE IS " + balance);
    }
}

class SavingsAccount extends Account {
    private double interestrate;

    public SavingsAccount(String accountnumber , String accountholder , double balance){
        super(accountnumber, accountholder, balance);
    }
}
```

```

        this.interestrate = interestrate;
    }

    @Override
    void withdraw(double amount){
        if(amount <= 0){
            System.out.println("with draw amount is not possible ");
        }else if (amount >= balance){
            System.out.println("it has insufficient balance");
        }else{
            balance -= amount;
            System.out.println("withdraw is " + amount );
        }
    }

    @Override
    void calinterest(){
        double interest = balance *interestrate;
        balance += interest;
        System.out.println("interest is credited" + interest);
    }
}

class CurrentAccount extends Account{
    private double overdraftlimit;

    public CurrentAccount(String accountnumber , String accountholder , double balance){
        super(accountnumber, accountholder, balance);
        this.overdraftlimit = overdraftlimit;
    }

    @Override
    void withdraw(double amount){
        if(amount <= 0)
            System.out.println("with draw amount is not possible ");
        else if (amount > balance + overdraftlimit){
            System.out.println("it has insufficient balance and the limited is done");
        }else{
            balance -= amount;
            System.out.println("withdraw is " + amount );
        }
    }

    @Override
    void calinterest(){
        double charge = 50.0;
        balance += charge;
        System.out.println("maintainence charge is credited" + charge);
    }
}

public class LabProgram1 {
    public static void main(String[] args) {
        CurrentAccount ca = new CurrentAccount("UB 24311", "kumar", 4500);
        SavingsAccount sa = new SavingsAccount("sb 24533", "heamnth", 5000);

        System.out.println(" initial account details");
        ca.show();
        System.out.println("");
        sa.show();

        System.out.println("\ntransaction details");
        ca.deposit(2000);
    }
}

```

```

        ca.withdraw(2500);
        ca.calinterest();

        sa.withdraw(3000);
        sa.calinterest();
        sa.transfer(ca, 1000);

        System.out.println("\nfinal account details");
        ca.show();
        System.out.println("");
        sa.show();
    }
}

```

6. Sample Output

```

initial account details
ACCOUNT NUMBER IS UB 24311
ACCOUNT HOLDER IS kumar
BALANCE IS 4500.0

ACCOUNT NUMBER IS sb 24533
ACCOUNT HOLDER IS heamnth
BALANCE IS 5000.0

transaction details
the deposited amount is 2000.0
withdraw is 2500.0
maintainence charge is credited50.0
withdraw is 3000.0
interest is credited0.0
transfer of 1000.0 from sb 24533 to UB 24311---
withdraw is 1000.0
the deposited amount is 1000.0

final account details
ACCOUNT NUMBER IS UB 24311
ACCOUNT HOLDER IS kumar
BALANCE IS 5050.0

ACCOUNT NUMBER IS sb 24533
ACCOUNT HOLDER IS heamnth
BALANCE IS 1000.0

```